

Jamia Millia Islamia

A Central University

DSC-503: COMPUTER NETWORKS

MR. HANNAN MANSOOR

DEPARTMENT OF COMPUTER ENGINEERING

JAMIA MILLIA ISLAMIA

UNIT- 2: THE DATA LINK LAYER

- Data Link Layer design issues,
- Framing,
- Error Detection and Correction,
- Flow control Protocols, Stop and Wait protocol, Sliding - window flow control,
- Error control, stop and wait ARQ, Go-back-N, Selective repeat ARQ,
- Examples of Data link Protocols- HDLC.

© HANNAN MANSOOR



DLL: INTRODUCTION

- Communication at the data-link layer is **node-to-node**.
- When a packet is travelling in the Internet, the data-link layer of a node (host or router) is responsible for delivering a datagram to the next node in the path.
- The data-link layer is located **between** the physical and the network layers.
- The first service provided by the data-link layer is **framing**.
- The data-link layer at each node needs to **encapsulate the datagram** (packet received from the network layer) in a **frame** before sending it to the next node.

© HANNAN MANSOOR

DATA LINK LAYER

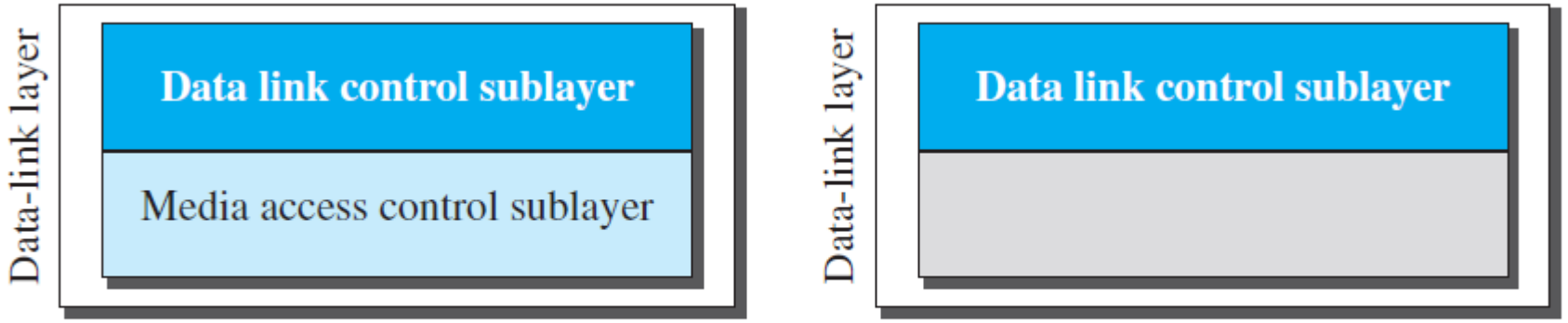
- FLOW CONTROL
  - Flow control refers to a set of procedures used to restrict the amount of data that the sender can send before waiting for acknowledgment.
  - Flow control coordinates the amount of data that can be sent before receiving an acknowledgment and is one of the most important duties of the data link layer.
- ERROR CONTROL
  - Error control is both error detection and error correction.
  - It allows the receiver to inform the sender of any frames lost or damaged in transmission and coordinates the retransmission of those frames by the sender.
  - In the data link layer, the term error control refers primarily to methods of error detection and retransmission.

DATA LINK LAYER

- CONGESTION CONTROL
- Although a link may be congested with frames, which may result in frame loss, most data-link-layer protocols do not directly use a congestion control to alleviate congestion, although some wide-area networks do.

DATA LINK LAYER: TWO SUBLAYER

- Data link control (DLC) and
- Media access control (MAC).



a. Data-link layer of a broadcast link

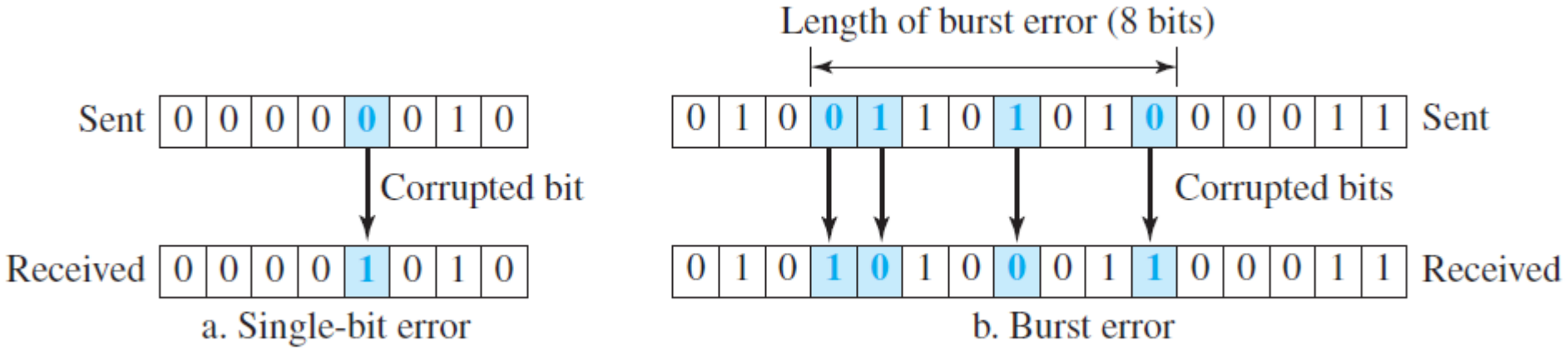
b. Data-link layer of a point-to-point link

ERROR DETECTION AND CORRECTION

- Data can be corrupted during transmission.
- Some applications require that errors be detected and corrected.
- TYPES OF ERRORS:
- Whenever bits flow from one point to another, they are subject to unpredictable changes because of interference.
- This interference can change the shape of the signal.
- The term **single-bit error** means that only 1 bit of a given data unit (such as a byte, character, or packet) is changed from 1 to 0 or from 0 to 1.
- The term **burst error** means that 2 or more bits in the data unit have changed from 1 to 0 or from 0 to 1.

© HANNAN MANSOOR

SINGLE BIT AND BURST ERROR



SINGLE BIT AND BURST ERROR

- The term **single-bit error** means that only 1 bit of a given data unit (such as a byte, character, or packet) is changed from 1 to 0 or from 0 to 1.
- The term **burst error** means that 2 or more bits in the data unit have changed from 1 to 0 or from 0 to 1.

- A burst error is more likely to occur than a single-bit error because the duration of the noise signal is normally longer than the duration of 1 bit, which means that when noise affects data, it affects a set of bits.
- The number of bits affected depends on the data rate and duration of noise.

© HANNAN MANSOOR

REDUNDANCY

- The central concept in detecting or correcting errors is **redundancy**.
- These redundant bits are **added by the sender** and **removed by the receiver**.
- Their presence allows the receiver to **detect or correct** corrupted bits.
- To detect or correct errors, we need to send extra (redundant) bits with data.

© HANNAN MANSOOR

REDUNDANCY

- In **error detection**, we are only looking to see if any error has occurred.
  - The answer is a simple **yes or no**.
  - We are not even interested in the number of corrupted bits.
- In **error correction**, we need to know the exact number of bits that are corrupted and, more importantly, their location in the message.
  - The **number of errors** and the **size of the message** are important factors.
  - If we need to correct a single error in an 8-bit data unit, we need to consider eight possible error locations;
  - if we need to correct two errors in a data unit of the same size, we need to consider 28 (permutation of 8 by 2) possibilities.

© HANNAN MANSOOR

BLOCK CODING

- In **block coding**, we divide our message into blocks, each of k bits, called **data-words**.
- We add **r redundant bits** to each block to make the **length n = k + r**.
- The resulting **n-bit blocks** are called **codewords**.
- For the moment, it is important to know that we have a set of **data-words**, each of **size k**, and a set of **codewords**, each of **size of n**.
- With k bits, we can create a combination of 2^k data-words; with n bits, we can create a combination of 2^n code-words.
- Since n > k, the number of possible codewords is larger than the number of possible data-words

© HANNAN MANSOOR

Error Detection

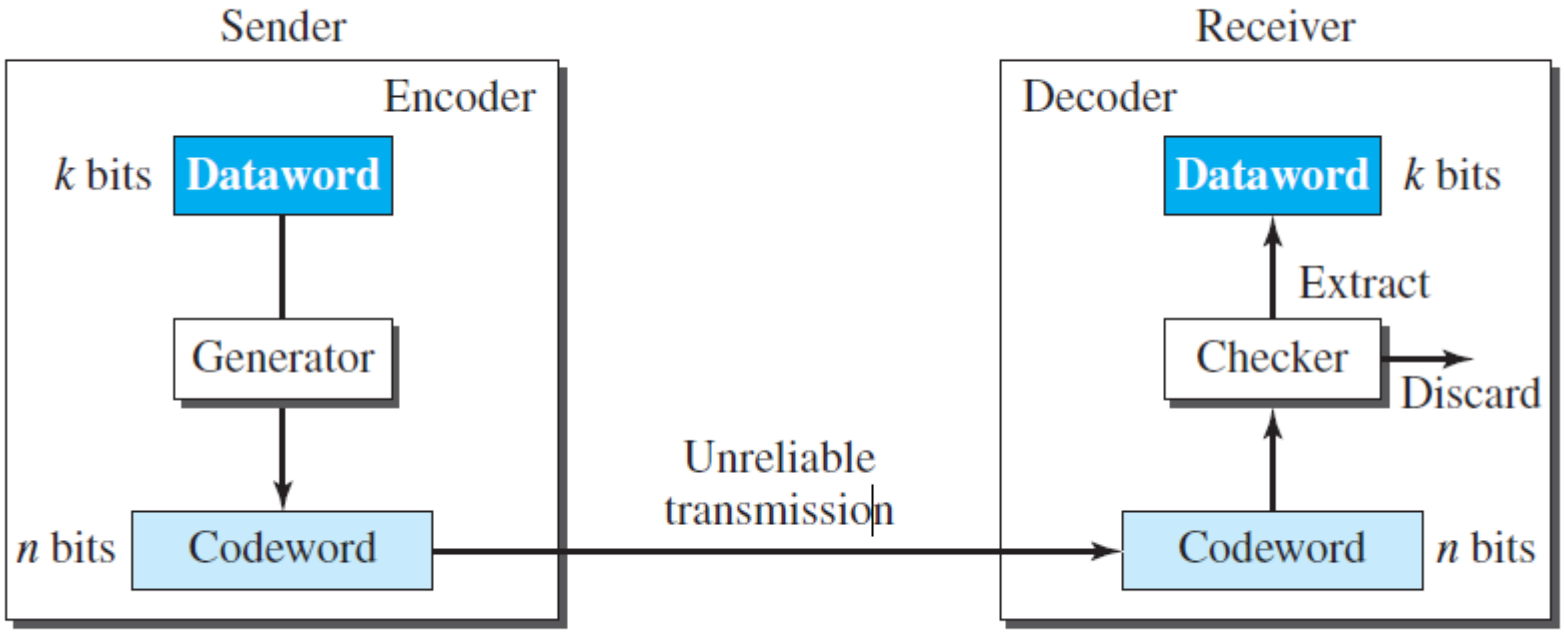


***Fig**: Process of error detection in block coding*

Example:

- Let us assume that k = 2 and n = 3. Table shows the list of data-words and code-words.
- The receiver receives 011. It is a valid codeword. The receiver extracts the data-word 01 from it.
- The codeword is corrupted during transmission, and 111 is received (the leftmost bit is corrupted).This is not a valid codeword and is discarded.
- The codeword is corrupted during transmission, and 000 is received (the right two bits are corrupted). This is a valid codeword. The receiver incorrectly extracts the data-word 00. Two corrupted bits have made the error undetectable

Data-word	Codeword
00	000
01	011
10	101
11	110

Hamming Distance

- The Hamming distance between two words (of the same size) is the number of differences between the corresponding bits.
- Hamming distance between two words x and y as d(x, y).
- Hamming distance between the received codeword and the sent codeword is the number of bits that are corrupted during transmission.
- Example:
- codeword 00000 is sent and 01101 is received,
- 3 bits are in error and the Hamming distance between the two is d(00000, 01101) = 3.

© HANNAN MANSOOR

Example: Hamming distance

- Let us find the Hamming distance between two pairs of words:
- The hamming distance d(000,011) is 2 because
- (000 $\oplus$ 011) is 011 (two 1s).
- The hamming distance d(10101,11110) is 3 because
- (10101,11110) is 01011(three 1s).

© HANNAN MANSOOR

Minimum Hamming Distance for Error Detection



- In a set of codewords, the minimum Hamming distance is the smallest Hamming distance between all possible pairs of codewords.
- Now let us find the minimum Hamming distance in a code if we want to be able to detect up to ***s*** errors.
- If ***s*** errors occur during transmission, the Hamming distance between the sent codeword and received codeword is ***s***.

© HANNAN MANSOOR

Parity-Check Code

EVEN PARITY

© HANNAN MANSOOR

PARITY

- Perhaps the most familiar **error-detecting code** is the parity-check code.
- This code is a **linear block code**.
- In this code, a **k-bit data-word** is changed to an **n-bit codeword** where **n = k + 1**.
- The extra bit, called the parity bit, is selected to make **the total number of 1s in the codeword even**.
- Although some implementations specify an odd number of 1s, we discuss the even case.

© HANNAN MANSOOR

HAMMING DISTANCE FOR PARITY

- The minimum Hamming distance for this category is dmin = 2, which means that the code is a single-bit error-detecting code.
- The code in Table is also a parity-check code with k = 4 and n = 5.

© HANNAN MANSOOR

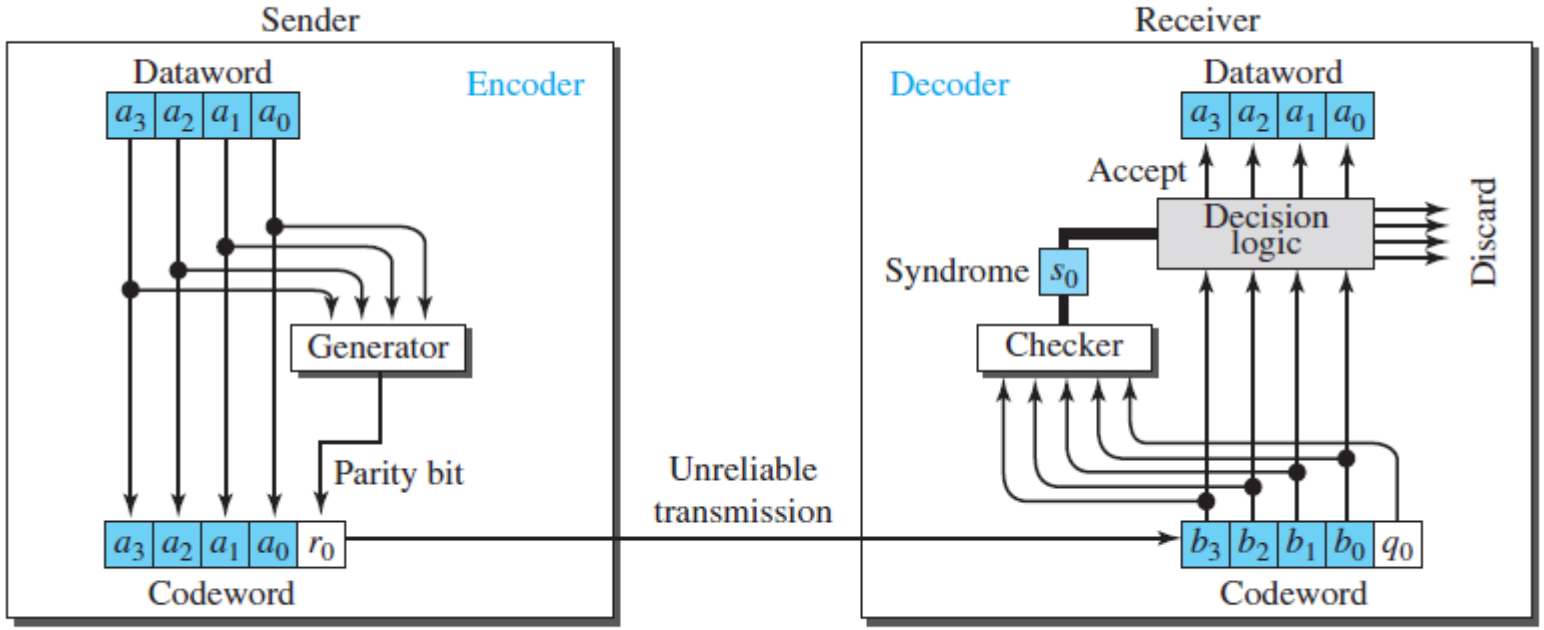
SIMPLE PARITY-CHECK CODE C(5,4)

Dataword	Codeword	Dataword	Codeword
0000	00000	1000	10001
0001	00011	1001	10010
0010	00101	1010	10100
0011	00110	1011	10111
0100	01001	1100	11000
0101	01010	1101	11011
0110	01100	1110	11101
0111	01111	1111	11110

TABLE: Simple parity check code C(5,4)

© HANNAN MANSOOR

Encoder and decoder for simple parity-check code



PARITY CHECK METHOD

- The calculation is done in **modular arithmetic**.
- The encoder uses a generator that takes a copy of a 4-bit data-word (a0, a1, a2, and a3) and generates a parity bit r0.
- The data-word bits and the parity bit create the 5-bit codeword.
- The parity bit that is added makes the number of 1s in the codeword even.
- This is normally done by adding the 4 bits of the data-word (modulo-2); the result is the parity bit.

$$r0 = a3 + a2 + a1 + a0$$

- If the number of 1s is even, the result is 0;
- if the number of 1s is odd, the result is 1.
- In both cases, the total number of 1s in the codeword is even.

© HANNAN MANSOOR



HAMMING CODE

- A category of error-correcting codes called Hamming codes.
- These codes were originally designed with dmin = 3, which means that they can detect up to two errors or correct one single error.
- First let us find the relationship between **n** and **k** in a Hamming code.
- We need to choose an integer $\mathbf{m} \geq \mathbf{3}$.
- The values of **n** and **k** are then calculated from mas $\boldsymbol{n} = \mathbf{2}^{\boldsymbol{m}} - \mathbf{1}$ and $\mathbf{k} = \boldsymbol{n} - \boldsymbol{m}$.
- The number of Check bits r=m.

PARITY CHECK METHOD FOR HAMMING CODE

- Let us take an input of 11 bits k = 00110001110
- Block diagram/ code-word $\boldsymbol{n = 2^m - 1}$
- Data-word/message bits (k)=11; $\boldsymbol{k = 2^m - m - 1}$
- Not of parity bits= $\boldsymbol{m = n - k}$ or$\boldsymbol{2^m \geq K + m + 1}$
- The minimum distance $\boldsymbol{d_{min} = 3}$
- If m=4, then n=15 and k=11.
- Hamming code (15,11) with dmin>=3

© HANNAN MANSOOR

Step 01: Make a 16 bit space

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

© HANNAN MANSOOR

S2: Parity bits for the sequence

0	1 P1	2 P2	3
4 P3	5	6	7
8 P4	9	10	11
12	13	14	15

Positions for parity = 1,2,4 and 8 (using $2^n, 2^0 = 1, 2^1 = 2, 2^2 = 4, 2^3 = 8.$

S3: insert word code (00110001110) and rename parity as their position.

0	1	2	3
	P1	P2	0
4	5	6	7
P4	0	1	1
8	9	10	11
P8	0	0	0
12	13	14	15
1	1	1	0

Positions for parity = 1,2,4 and 8 (using $2^n, 2^0 = 1, 2^1 = 2, 2^2 = 4, 2^3 = 8$.

© HANNAN MANSOOR

S4: Set Parity P1

- Count no of 1s in even column 2nd and 4th Column and set even parity at P1.
- P1=0

	C1	C2	C3	C4
R1	0	1 P1	2 P2	3 0
R2	4 P4	5 0	6 1	7 1
R3	8 P8	9 0	10 0	11 0
R4	12 1	13 1	14 1	15 0

S4: Set Parity P2

- Count no of 1s in 3rd and 4th column and set even parity at P2.
- P2=1

	C1	C2	C3	C4
R1	0	1 **P1=0**	2 **P2**	3 **0**
R2	4 **P4**	5 **0**	6 **1**	7 **1**
R3	8 **P8**	9 **0**	10 **0**	11 **0**
R4	12 **1**	13 **1**	14 **1**	15 **0**

© HANNAN MANSOOR

S4: Set Parity P4

- Count no of 1s in 2nd and 4th Row and set even parity at P4.
- P4=1

	C1	C2	C3	C4
R1	0	1 **P1=0**	2 **P2=1**	3 **0**
R2	4 **P4**	5 **0**	6 **1**	7 **1**
R3	8 **P8**	9 **0**	10 **0**	11 **0**
R4	12 **1**	13 **1**	14 **1**	15 **0**

© HANNAN MANSOOR

S4: Set Parity P8

- Count no of 1s in 3rd and 4th Row and set even parity at P8.
- P8=1

	C1	C2	C3	C4
R1	0	1 **P1=0**	2 **P2=1**	3 **0**
R2	4 **P4=1**	5 **0**	6 **1**	7 **1**
R3	8 **P8**	9 **0**	10 **0**	11 **0**
R4	12 **1**	13 **1**	14 **1**	15 **0**

© HANNAN MANSOOR

The message generated = (0010-1011-1000-1110)

- For Column 1 and Row 1 count value no of 1s in the and we get it as 0.

	C1	C2	C3	C4
R1	0 **0**	1 **0**	2 **1**	3 **0**
R2	4 **1**	5 **0**	6 **1**	7 **1**
R3	8 **1**	9 **0**	10 **0**	11 **0**
R4	12 **1**	13 **1**	14 **1**	15 **0**

The message generated is (0010-1011-1000-1110)

- For error checking insert an error in the message and at a location then use the coding method for checking error.
- The message that is transmitted is **(0010-1011-1000-1110)** but the message received is **(0010-1011-1001-1110)**

© HANNAN MANSOOR

The message received = (0010-1011-1001-1110)

- The message received has an error, detect it using hamming code and correct it.

	C1	C2	C3	C4
R1	0 **0**	1 **0**	2 **1**	3 **0**
R2	4 **1**	5 **0**	6 **1**	7 **1**
R3	8 **1**	9 **0**	10 **0**	11 **1**
R4	12 **1**	13 **1**	14 **1**	15 **0**

Find P1 for the received message

- P1 for received message = 1 as compared to send P1=0
- Error in P1 = **Yes**

	C1	C2	C3	C4
R1	0 **0**	1 **0**	2 **1**	3 **0**
R2	4 **1**	5 **0**	6 **1**	7 **1**
R3	8 **1**	9 **0**	10 **0**	11 **1**
R4	12 **1**	13 **1**	14 **1**	15 **0**

© HANNAN MANSOOR

Find P2 for the received message

- P2 for received message = 0 as compared to send P2=1
- Error in P2 = **Yes**

	C1	C2	C3	C4
R1	0 **0**	1 **0**	2 **1**	3 **0**
R2	4 **1**	5 **0**	6 **1**	7 **1**
R3	8 **1**	9 **0**	10 **0**	11 **1**
R4	12 **1**	13 **1**	14 **1**	15 **0**

Find P4 for the received message

- P4 for received message = 1 as compared to send P4=1
- Error in P4 = **No**

	C1	C2	C3	C4
R1	0 **0**	1 **0**	2 **1**	3 **0**
R2	4 **1**	5 **0**	6 **1**	7 **1**
R3	8 **1**	9 **0**	10 **0**	11 **1**
R4	12 **1**	13 **1**	14 **1**	15 **0**

© HANNAN MANSOOR

Find P8 for the received message

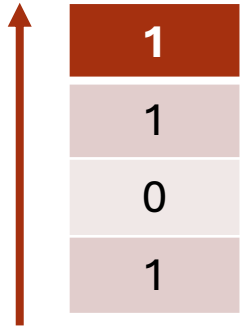
- P8 for received message = 0 as compared to send P8=1
- Error in P8 = **Yes**

	C1	C2	C3	C4
R1	0 **0**	1 **0**	2 **1**	3 **0**
R2	4 **1**	5 **0**	6 **1**	7 **1**
R3	8 **1**	9 **0**	10 **0**	11 **1**
R4	12 **1**	13 **1**	14 **1**	15 **0**

© HANNAN MANSOOR

Check Parity transmitted and received

- Parity for message that is transmitted = P1, P2, P4 & P8 (0111)
- **(0010-1011-1000-1110)**
- Parity for the message received = P1, P2, P4 & P8 (1010)
- **(0010-1011-1001-1110)**
- Error in parity(Y,Y,N,Y)= (1101)
- We get the value =11 i.e. the location of error.



- Flip the bit at location and get the correct message.

© HANNAN MANSOOR

CYCLIC CODES

- Cyclic codes are special linear block codes with one extra property. In a cyclic code, if a codeword is cyclically shifted (rotated), the result is another codeword.
- For example, if 1011000 is a codeword and we cyclically left-shift, then 0110001 is also a codeword.

© HANNAN MANSOOR

CYCLIC REDUNDANCY CHECK

- We can create cyclic codes to correct errors.
- a subset of cyclic codes called the cyclic redundancy check (CRC), which is used in networks such as LANs and WANs.

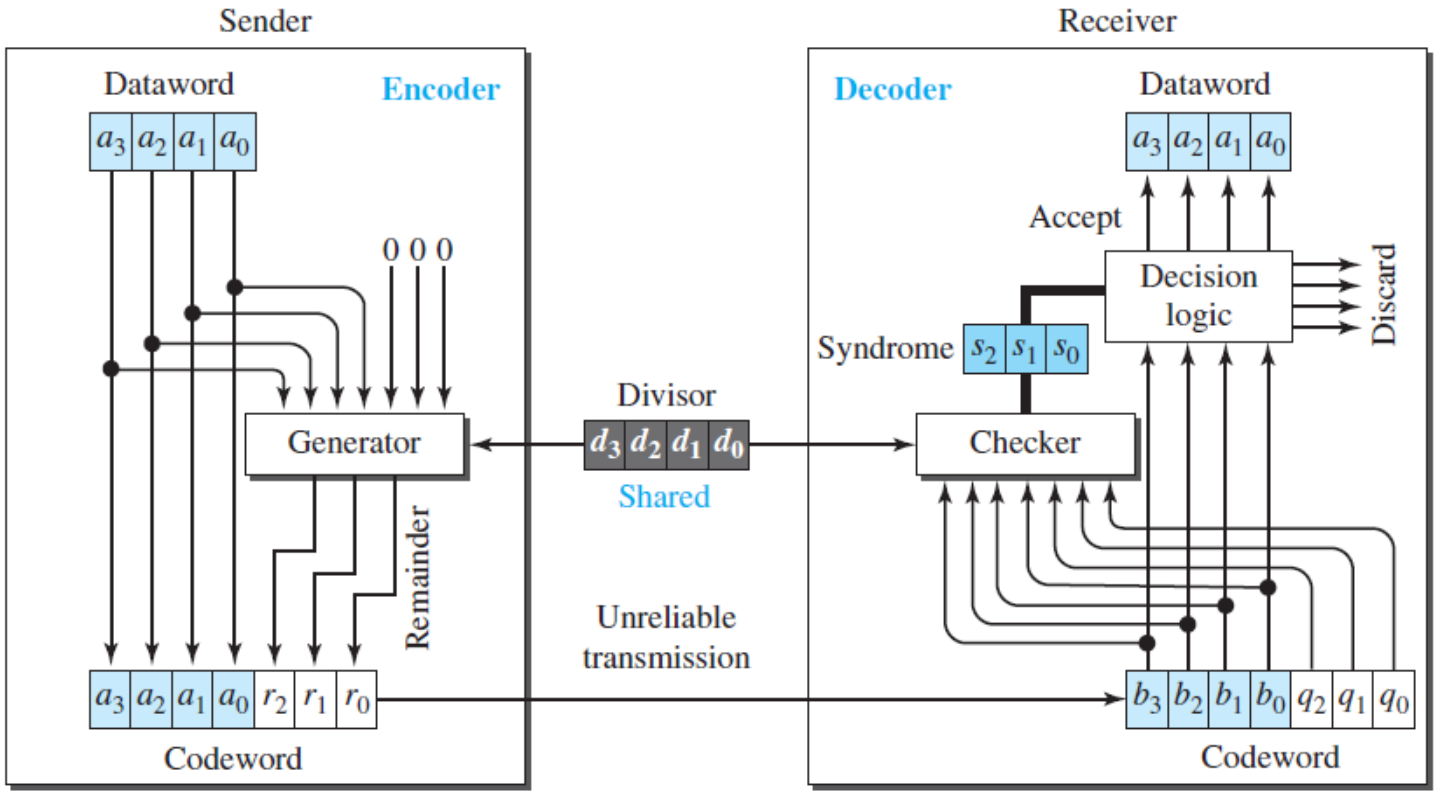
© HANNAN MANSOOR

A CRC CODE WITH C(7,4)

Dataword	*Codeword*	*Dataword*	*Codeword*
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

© HANNAN MANSOOR

CRC encoder and decoder



CRC ENCODER

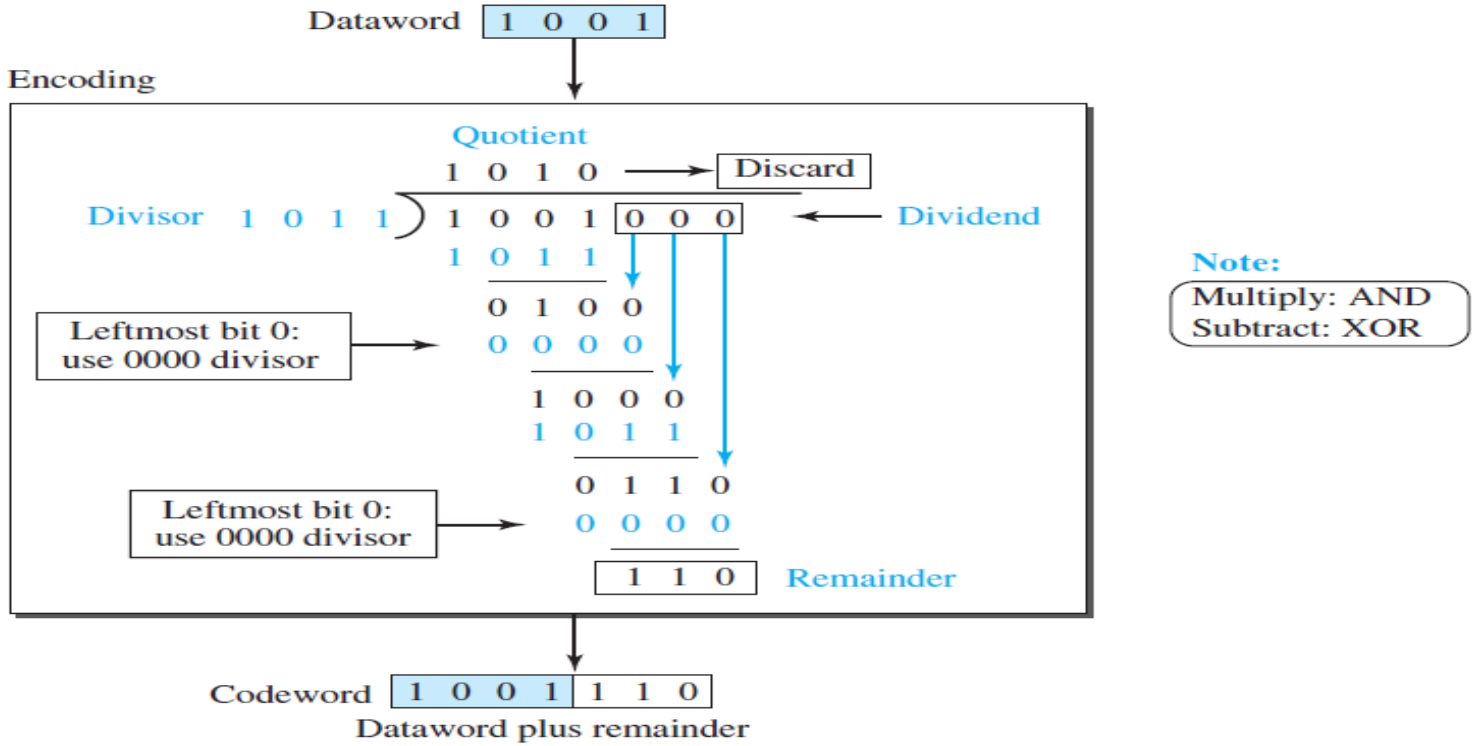
- In the encoder, the dataword has k bits (4 here); the codeword has n bits (7 here).
- The size of the dataword is augmented by adding n − k (3 here) 0s to the right-hand side of the word.
- The n-bit result is fed into the generator.
- The generator uses a divisor of size n − k + 1 (4 here), predefined and agreed upon.
- The generator divides the augmented dataword by the divisor (modulo-2 division).
- The quotient of the division is discarded;the remainder (r2r1r0) is appended to the dataword to create the codeword.

© HANNAN MANSOOR

CRC DECODER

- The decoder receives the codeword(possibly corrupted in transition).
- A copy of all n bits is fed to the checker, which is a replica of the generator.
- The remainder produced by the checker is a syndrome of n − k (3 here) bits, which is fed to the decision logic analyzer.
- The analyzer has a simple function. If the syndrome bits are all 0s, the 4 leftmost bits of the codeword are accepted as the dataword (interpreted as no error); otherwise, the 4 bits are discarded (error).

DIVISION IN CRC ENCODER





CHECKSUM

- The checksum is used in the Internet by several protocols although not at the data link layer.
- Like linear and cyclic codes, the checksum is based on the concept of redundancy.

© HANNAN MANSOOR

CHECKSUM EXAMPLE - 01

- Suppose our data is a list of five 4-bit numbers that we want to send to a destination.
- In addition to sending these numbers, we send the sum of the numbers.
- if the set of numbers is (7, 11, 12, 0, 6),
- we send (7, 11, 12,0,6,36), where 36 is the sum of the original numbers.
- The receiver adds the five numbers and compares the result with the sum.
- If the two are the same, the receiver assumes no error, accepts the five numbers, and discards the sum.
- there is an error somewhere and the data are not accepted.

CS EXAMPLE - 2

- We can make the job of the receiver easier if we send the negative (complement) of the sum, called the checksum.
- In this case, we send (7, 11, 12,0,6, -36).
- The receiver can add all the numbers received (including the checksum).
- If the result is 0, it assumes no error; otherwise, there is an error.

© HANNAN MANSOOR

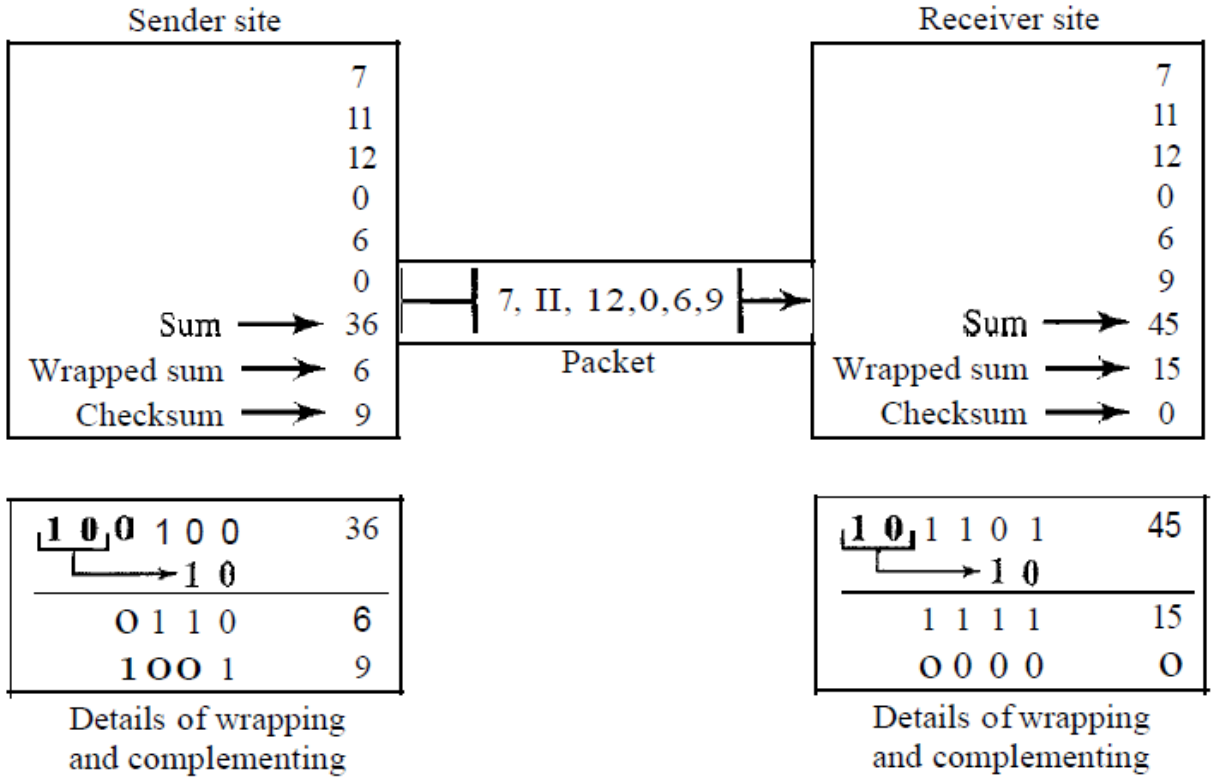
CS EXAMPLE - 3

- How can we represent the number 21 in one's complement arithmetic using only four bits?
- **Solution**:
- The number 21 in binary is 10101 (it needs five bits).
- We can wrap the leftmost bit and
- add it to the four rightmost bits.
- We have (0101 + 1) = 0110 or 6.

CS EXAMPLE - 4

- How can we represent the number -6 in one's complement arithmetic using only four bits?
- **Solution**:
- In one's complement arithmetic, the negative or complement of a number is found by inverting all bits.
- Positive 6 is 0110; negative 6 is 100I.
- If we consider only unsigned numbers, this is 9.
- In other words, the complement of 6 is 9.
- Another way to find the complement of a number in one's complement arithmetic is to subtract the number from 2n - I (16 - 1 in this case).

CS EXAMPLE - 5


Sender site
7
11
12
0
6
0
Sum → 36
Wrapped sum → 6
Checksum → 9
7, II, 12,0,6,9
Packet
Receiver site
7
11
12
0
6
9
Sum → 45
Wrapped sum → 15
Checksum → 0
1 0 0 1 0 0 36
1 0
0 1 1 0 6
1 0 0 1 9
Details of wrapping and complementing
1 0 1 1 0 1 45
1 0
1 1 1 1 15
0 0 0 0 0
Details of wrapping and complementing

Internet Checksum

- Traditionally, the Internet has used a 16-bit checksum. The sender and the receiver follow the steps depicted in Table

PROCEDURE TO CALCULATE THE CHECKSUM

Sender site:

1. The message is divided into 16-bit words.
2. The value of the checksum word is set to 0.
3. All words including the checksum are added using one's complement addition.
4. The sum is complemented and becomes the checksum.
5. The checksum is sent with the data.

The receiver uses the following steps for error detection.

Receiver site:

1. The message (including checksum) is divided into 16-bit words.
2. All words are added using one's complement addition.
3. The sum is complemented and becomes the new checksum.
4. **If** the value of checksum is 0, the message is accepted; otherwise, it is rejected.

CS EXAMPLE - 5

- Let us calculate the checksum for a text of 8 characters ("Forouzan"). The text needs to be divided into 2-byte (l6-bit) words.
- We use ASCII (see Appendix A) to change each byte to a 2-digit hexadecimal number.
- For example, F is represented as Ox46 and 0 is represented as Ox6F.
- the value of partial sum for the first column is Ox36.
- We keep the rightmost digit (6) and insert the leftmost dight (3) as the carry in the second column.
- The process is repeated for each column.

© HANNAN MANSOOR

Example

1	0	1	3	Carries
4	6	6	F	(Fo)
7	2	6	F	(ro)
7	5	7	A	luz)
6	1	6	E	(an)
0	0	0	0	Checksum (initial)
8	F	C	6	Sum (partial)
			1	
8	F	C	7	Sum
7	0	3	8	Checksum (to send)

a. Checksum at the sender site

1	0	1	3	Carries
4	6	6	F	IFo)
7	2	6	F	(ro)
7	5	7	A	(uz)
6	1	6	E	(an)
7	0	3	8	Checksum (received)
F	F	F	E	Sum (partial)
			1	
F	F	F	F	Sum
0	0	0	0	Checksum (new}

b. Checksum at the receiver site



Refer Class Notes and Recommended Books for further Reading

Reference Books

- Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer networks* (5th ed.). Prentice Hall.
- Forouzan, B. A. (2012). *Data communications and networking* (4th ed.). McGraw-Hill.
- Kurose, J. F., & Ross, K. W. (2021). Computer networking: A top-down approach (8th ed.). Pearson.
- Peterson, L. L., & Davie, B. S. (2021). Computer networks: A systems approach (6th ed.). Morgan Kaufmann.
- Miller, M. A. (1992). Introduction to Digital and Data Communications. United States: West Publishing Company.
- Haykin, Simon, 3Rd Ed. (2008). Communication Systems India: Wiley India Pvt. Limited.
- Frenzel. (2016). Principles of Electronic Communication Systems. United Kingdom: McGraw-Hill Education.



Thank You